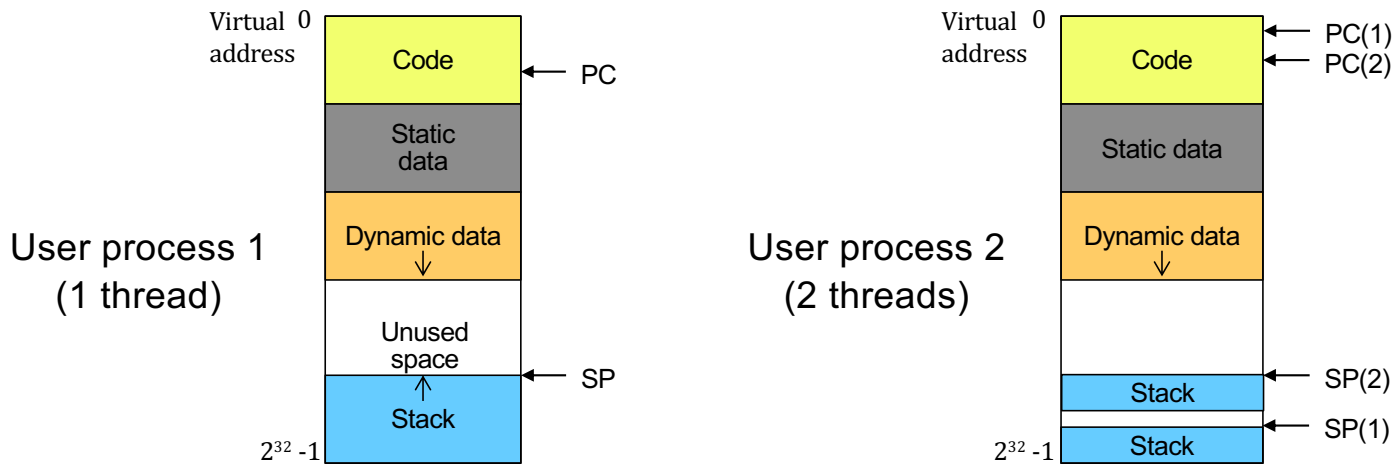
CSCC69 Operating Systems

Today

- Processes & threads
- Possibly start with memory management

Recap processes and threads



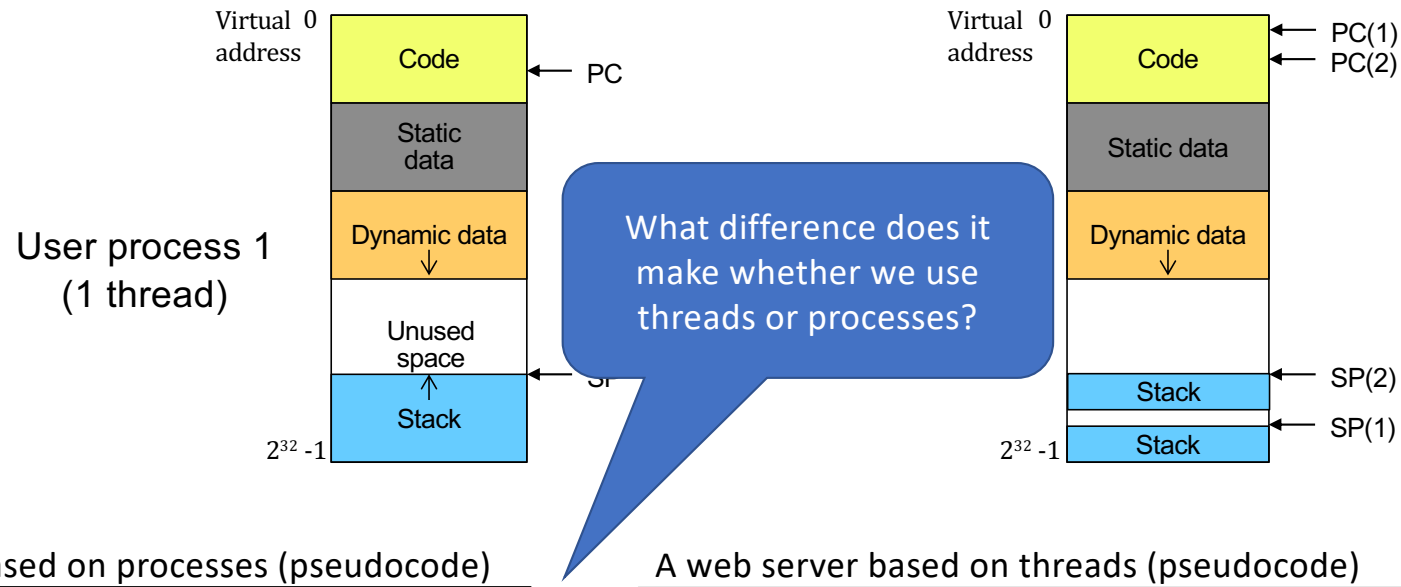
Why do we care about having multiple processes and/or multiple threads?

⇒Parallelism/concurrency

⇒Multiple users (multiple processes)

⇒Encapsulating work

Example use case: a web server



A web server based on processes (pseudocode)

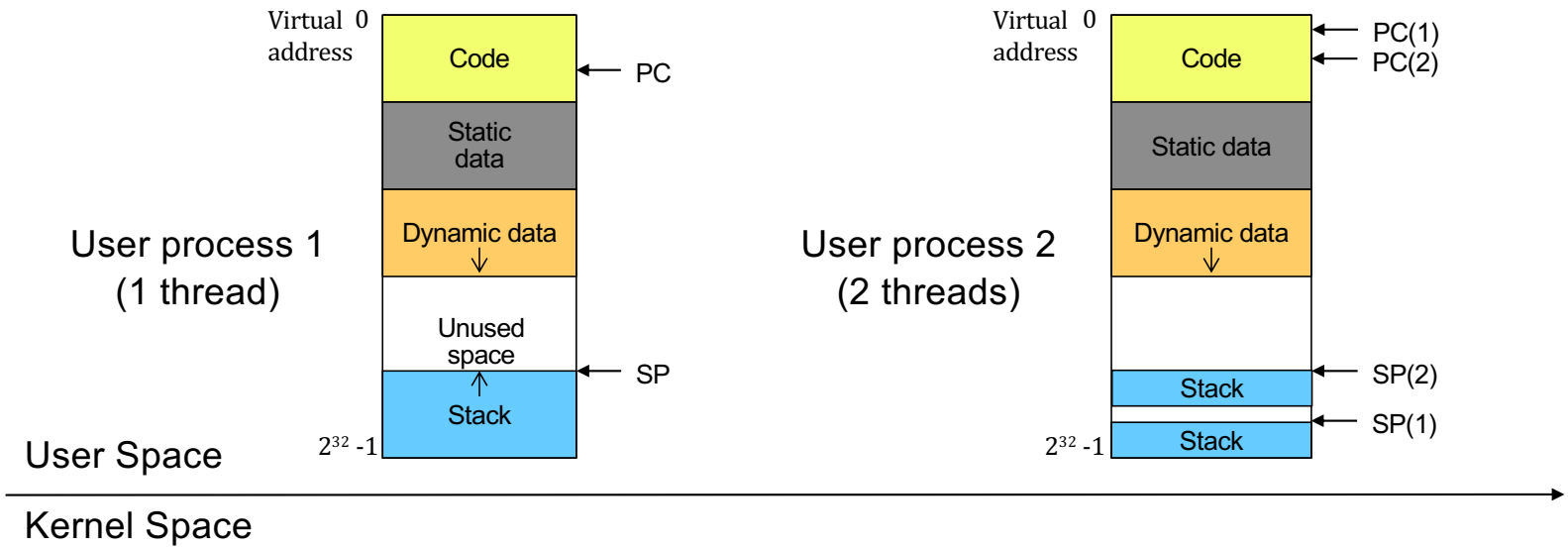
```
while (1) {
  int sock = accept();
  if ((child_pid = fork()) == 0) {
    // Handle client request
  } else {
    // Close socket
  }
}
```

A web server based on threads (pseudocode)

```
while(1) {
    int sock = accept();
    pthread_create(…, handle_request, &sock);
}

void handle_request(int *sock) {
    // Handle client request
}
```

Processes and threads – Kernel data structures



For each process:

Process Control Block (PCB)

- Pid (process ID) and PPID (Parent process)
- Address space (coming soon)
- Open files (coming later)
- Others

(`struct process` in Pintos)

For each thread:

Thread Control Block (TCB)

- Tid (thread ID)
- State (running, ready,…)
- Registers (PC, SP)
- Pointer to PCB

(`struct thread` in Pintos)

Process creation versus **user thread** creation

Much less overhead to create a thread than a process!

`Process creation with fork()`

1. Create and initialize a new PCB
2. Create a new address space
3. Initialize the address space with a copy of parent address space
4. Initialize kernel resources to point to parent resources (e.g., open files)
5. Create a kernel thread (incl. TCB) and place that thread onto the ready queue

`Thread creation with pthread_create()`

1. Allocate & initialize Thread Control Block (TCB)
2. Allocate stack
3. Put func, args on stack
4. Put thread on ready list

Process communication overheads

Much less overhead for communication between threads!

- Communications between user threads in the same process?
  - Easy – shared memory

- Communication between different processes?
  - Pipes (same machine) or sockets (different machines)
  - Requires system calls to communicate

Context switching overheads of threads versus process

Less overhead for threads!

For both threads and processes:

- Save execution context (e.g. program counter and registers)
- Move TCB to ready queue

Only for processes:

- Change to a different address space (different memory mapping)
- Will see details later

Processes: A good but costly abstraction

- Creating a process is costly (space and time)
- Context switching is costly (time)
- Inter-process communication is costly (time)

✓ Then why use processes instead of threads?

Want to run a new program (fork + exec)

Processes are isolated from each other (no shared memory)

One processes crashing affects no other processes, thread crashing takes down corresponding process.

(Side note: what about `exec`?)

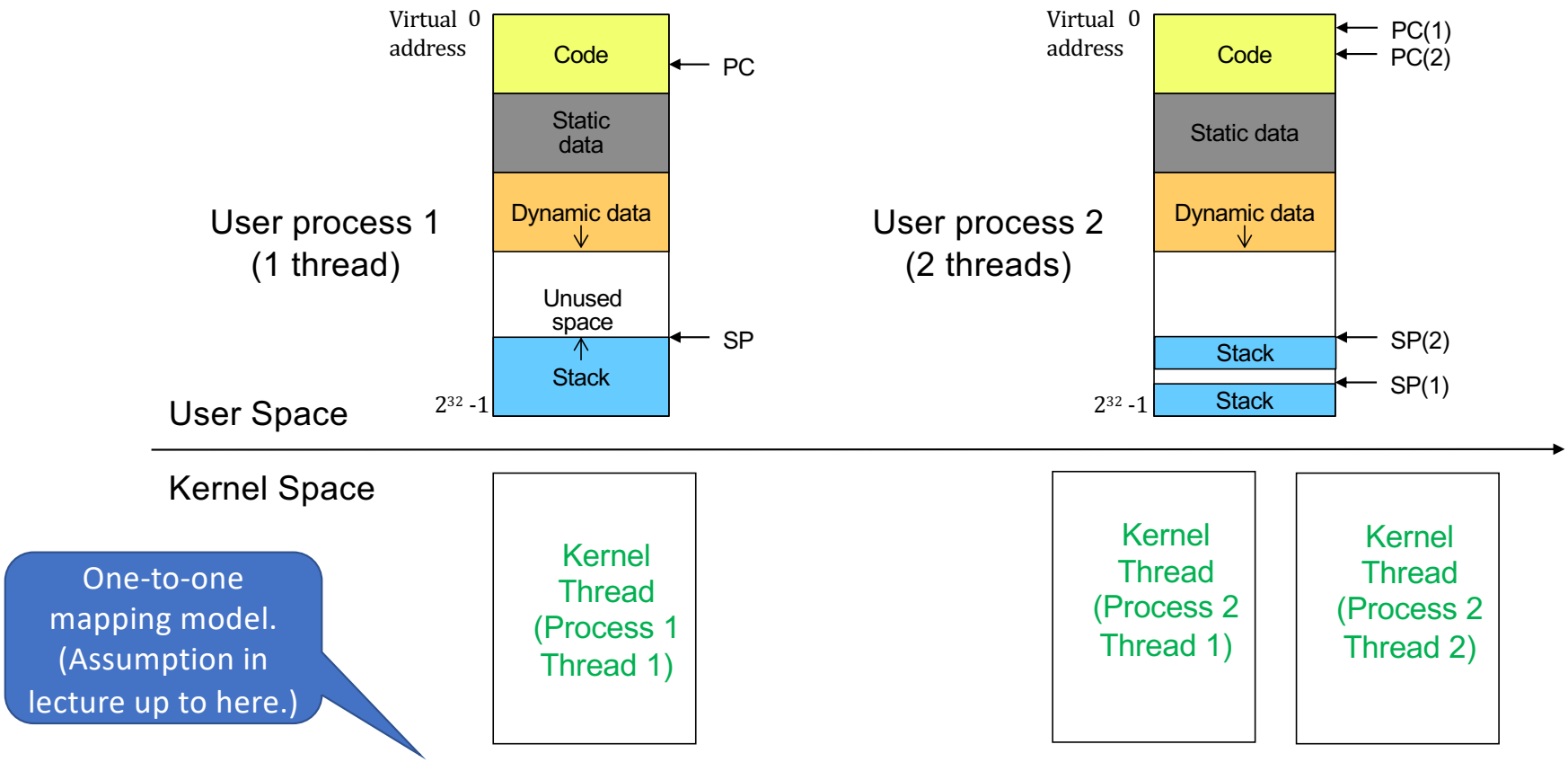
```
int exec(char *prog, char *argv[])
```

Actually, `exec` does not create a new process

Replaces image of process

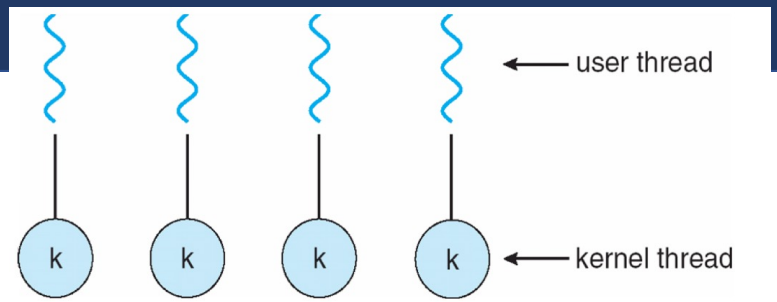
1. Stops the current process
2. Loads the program “prog” into the process’ address space
3. Initializes hardware context and args for the new program
4. Places the PCB onto the ready queue

How are user threads mapped to kernel threads? (Lecture 2 view)



- **Each user thread is mapped to a kernel thread (Linux Pthreads, Windows threads, Pintos).**
- The OS also has its own kernel threads or OS tasks.
- Kernel threads are the unit of scheduling in the OS.

One-to-one model



The kernel manages and schedules threads

- e.g Windows threads
- e.g POSIX pthreads `PTHREAD_SCOPE_SYSTEM` (only option in linux)
- e.g (new) Solaris lightweight processes (LWP)

All thread operations are managed by the kernel

✓ good for scheduling

✓ bad for speed

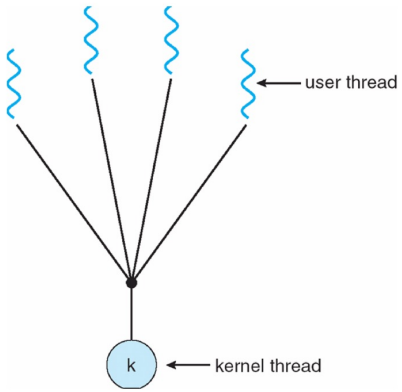
Multithreading models

- So far we assumed:
  - One-to-one model
    Kernel-level threads (a.k.a native threads)
- Others:
  - Many-to-one model
    User-level threads (a.k.a green threads)
  - Many-to-many model
    Hybrid threads (a.k.a n:m threading)

Many-to-one model

One kernel thread per process thread managemen
and scheduling is delegated to a library (user level)

- e.g Early Java threads



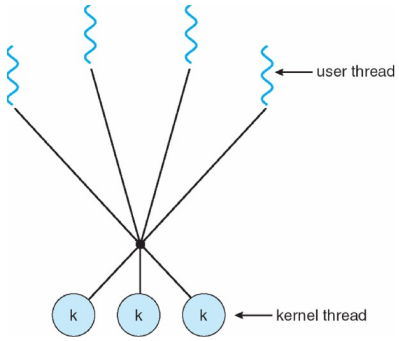
The kernel is not involved

✓ Very lightweight and fast

✓ All threads can be blocked if one of them is waiting for an event

✓ Cannot be scheduled on multiple cores

Many-to-many model



User threads implemented on kernel threads

- e.g (old) Solaris

Multiple kernel-level threads per process

Goal: Get the parallelism benefits of one-to-one without the per-thread overhead.

Downside: Complexity

- Need scheduling at two levels
- Need coordination between user and kernel level schedulers (or else kernel elve not aware of user level priorities)

Acknowledgments

- Some of the course materials and projects are from:
  - Youjip Won at Hanyang University.
  - Ryan Huang - teaching CS 318 at John Hopkins University •
  - Ben Pfaff - creator of Pintos and researcher at VMware Research Group •
  - David Mazière - teaching CS 140 at Stanford
  - Thierry Sans here at UTSC